React Context API

Learning Objectives

- Remove prop drilling from a React app using the Context API
- Reduce state complexity by moving local React state to the application level

Intro

As an application grows in size, **state** generally starts to become messy. Up until now, you've been using **prop drilling** to pass pieces of state down to the different components that need that state.

Q. Can you imagine what the Facebook code would look like if users and posts were prop drilled to every single component that might need access to the data?

Prop Drilling

Here's a simple example of an app that relies on prop drilling to pass data around.

This is already getting messy and it's only 4 components!

```
import postsArr from "./posts.js"
import usersArr from "./users.js"

function App() {
  const [posts, setPosts] = useState(postsArr)
  const [users, setUsers] = useState(usersArr)

  return (
    <SearchBar posts={posts} users={users} />
    <CreatePost posts={posts} setPosts={setPosts} />
    <Posts posts={posts} users={users} />
    <LeftSideBar users={users} />
  );
}
```

Ideal Solution

Here's something closer to how we'd like it to look.

We're not prop drilling anything, we just want to place our components into our app and let them each access the state they need.

But how do we do this? As it is right now, none of the components have any access to anything in state...

```
import postsArr from "./posts.js"
import usersArr from "./users.js"

function App() {
  const [posts, setPosts] = useState(postsArr)
  const [users, setUsers] = useState(usersArr)

  return (
    <SearchBar />
    <CreatePost />
    <Posts />
    <LeftSideBar />
  );
}
```

A State Wrapper!

With the **Context API**, we can wrap a special kind of react component around all of our components that need access to state.

Any component that renders as a **child** of that wrapper component, in this example `<MyContext>`, can be given access to anything we want!

There are a few things we need to do first in order to be able to use this Context component...

```
import postsArr from "./posts.js"
import usersArr from "./users.js"

function App() {
  const [posts, setPosts] = useState(postsArr)
  const [users, setUsers] = useState(usersArr)

  return (
    <MyContext>
      <SearchBar />
      <CreatePost />
      <Posts />
      <LeftSideBar />
    </MyContext>
  );
}
```

A new function: createContext

First, we import the **createContext** function from React.

Next, create a variable and set its value to be whatever is returned from running that function!

As a convention, we use PascalCase for the variable name since the **createContext** function will give us a special kind of react component.

We'll need this component later, so make sure to export it from the file you define it in. You could move it to its own file, but in this example we'll keep it in the App.jsx file.

https://react.dev/reference/react/createContext

```
import { createContext } from "react"

const MyContext = createContext()

function App() {
  return (
    <MyContext>
      <SearchBar />
      <CreatePost />
      <Posts />
      <LeftSideBar />
    </MyContext>
  );
}

export { MyContext, App }
```

Context Providers

Next, we attach an object to the **Context Provider** that will hold all of the state your application needs.

The `<Context.Provider>` component has a prop called `value` which accepts any valid JS data - a string, an array, an object, etc. The value that we provide is the value which the child components will be able to read via the Context API.

In this example, let's attach our `posts` and `users` states as an object to our **Context Provider**.

https://react.dev/reference/react/createContext#provider

```
import { createContext } from "react"
import postsArr from "./posts.js"
import usersArr from "./users.js"

const MyContext = createContext()

function App() {
  const [posts, setPosts] = useState(postsArr)
  const [users, setUsers] = useState(usersArr)

  return (
    <MyContext.Provider value={ { posts, users } }>
      <SearchBar />
      <CreatePost />
      <Posts />
      <LeftSideBar />
    </MyContext.Provider>
  );
}

export { MyContext, App }
```

A new hook: useContext

The final step is to pull in the state provided by the context into your individual components. Instead of prop drilling, we do this using the **useContext** hook.

You have to pass in the context you want to use, which is why we exported our **MyContext** from the **App.jsx** component.

Inside our `<Posts>` component, we can use the **useContext** hook to pull in anything we attached to the **value** prop of `<Context.Provider>`!

https://react.dev/reference/react/useContext

```
import { MyContext } from "../App.jsx"
import { useContext } from "react"

function Posts() {
  const context = useContext(MyContext)

  return (
    <>
      {context.posts.map(post => <Post post={post}/>)
    </>
  );
}
```

Context Needs State

Context is allows you to pass the value of a `state` around without prop-drilling. **It is NOT intended to replace state!**

We still need to use the state setter function to change the value of a state. A state's value might be controlled by a lower level component, which means we can pass the setter function in the <MyContext.Provider> too!

In this example, we have passed the setPosts function as an additional property of the object we are assigning to `value`.

```
import { createContext } from "react"
import postsArr from "./posts.js"
import usersArr from "./users.js"

const MyContext = createContext()

function App() {
  const [posts, setPosts] = useState(postsArr)
  const [users, setUsers] = useState(usersArr)

  return (
    <MyContext.Provider value={ { posts, users, setPosts } }>
      <SearchBar />
      <CreatePost />
      <Posts />
      <LeftSideBar />
    </MyContext.Provider>
  );
}

export { MyContext, App }
```

Context Needs State

It seems most logical that the <CreatePost> component will make use of the setPosts function. As before, we use `useContext` to access the values provided by MyContext.

If we create a new post, then we call setPosts and the posts state will be updated *(full implementation not shown here)*.

Other components needing the value of posts from the <MyContext.Provider> will receive the updated value of posts (e.g. the <Posts> component).

```
import { MyContext } from "../App.jsx"
import { useContext } from "react"

function CreatePost() {
  const context = useContext(MyContext)

  return (
    <form onSubmit={context.setPosts}>
        <label>New Post:
            <input type="text"></input>
        </label>
        <input type="submit" value="CREATE POST!"></input>
    </form>
  );
}
```

Teacher Live Demo

Your teacher will now demonstrate this

Morning Practice

Let's check this out using this app

Only do Part 1 in this practice repo

- **Remove prop-drilling by using the Context API**
- **You will need to use createContext and useContext**
- **Do this for the posts/setPosts state**

Do NOT do Part 2 yet!

Multiple Contexts?!

We can create as many **Context** components as we need for our app. We just have to make sure that all of the components that need the context are children of the **Context** at some level.

For simplicity, we can just create and export the **Contexts** in the **App.jsx** file.

For example, we may want to create a **Context each** for the user to select between light and dark mode (`ThemeContext`), and another for their data (`DataContext`).

```
import { createContext } from "react"
import postsArr from "./posts.js"
import usersArr from "./users.js"

const ThemeContext = createContext()
const DataContext = createContext()

function App() {
  const [theme, setTheme] = useState("light")
  const [posts, setPosts] = useState(postsArr)
  const [users, setUsers] = useState(usersArr)

  return (
    <ThemeContext.Provider value={theme}>
      <DataContext.Provider value={ { posts: posts, users: users } }>
        <SearchBar />
        <CreatePost />
        <Posts />
        <LeftSideBar />
      </DataContext.Provider>
    </ThemeContext.Provider>
  );
}

export { DataContext, ThemeContext, App }
```

Multiple Contexts?!

We can then call the useContext hook in each of the components which may need each piece of data from either or both of the **Contexts**.

```
import { DataContext, ThemeContext } from "../App.jsx"
import { useContext } from "react"

function Posts() {
  const dataContext = useContext(DataContext)
  const themeContext = useContext(ThemeContext)

  return (
    <div className={themeContext}>
      {dataContext.posts.map(post => <Post post={post}/>)
    </div>
  );
}
```

Afternoon Exercise

Follow the instructions for Part 1 in this practice repo:

https://github.com/boolean-uk/react-context-api

Do NOT do Part 2 yet!